A NOVEL EVOLUTIONARY GAME THEORETIC GREY WOLF OPTIMIZATION FOR OPTIMIZED EDGE COMPUTING AND TASK OFFLOADING IN 6G NETWORKS

Leon Changara Chemwor Odari[a,1,*], Davies Rene Segera[a]

[a]*Department of Electrical and Electronics Engineering, University of Nairobi, Nairobi, Kenya*

Abstract

6th generation (6G) networks coupled together with edge computing are set to transfrom the digital landscape including IoT and help enable innovative technologies like telemedicine for remote surgeries and holographic displays for immersive communication.However such applications often have lots of data that overwhelm their local processing and often have to trade off between energy and speed when it comes to processing.Offloading of tasks to nearby edge servers offer a solution but deciding which tasks to send, where to send them and how to manage edge resource allocations is challenging in a 6G environment.To address this, a new approach EGT-GWO is proposed, this approach combines evolutionary game theory principles on how the devices handle task offloading decisions decisions and grey wolf optimization to handle the resource allocations at the edge servers.This hybrid approach aims to provide real-time adaptation to fluctuating demands and network conditions of 6G.

Keywords: 6G Networks, Edge Computing, Task Offloading, Evolutionary Game Theory, Grey Wolf Optimization

1. INTRODUCTION

Wireless communication systems have evolved at a fast pace over the past couple of years,since the introduction of sms and mms brought about by 2G networks in the early 1990's to the colossal breakout of 3G networks in the 2000's which brought higher data speeds of up to 2Mbps on average making web surfing and early mobile applications a norm.4G came later in the 2010's revolutionizing the industry with technologies such as LTE(Long term Evolution) which led to streaming in HD and online mobile gaming.The low latency and faster speeds led to the abandonment of traditional broad-bands and in certain areas there was the rise of Iot(Internet of Things) and cloud application services. The release of 5G in early 2020 saw speeds of up to 10Gbps become a reality,enhanced mobile broadband (eMBB) and ultra-reliable Low-Latency communication(uRLLc) became game-changing technologies especially since they enhanced support for AR(augmented reality) and VR(virtual reality) and autonomous vehicles.However, with the escalation in the number of mobile devices(MD's) and Things ,many of the existing applications require real-time interactions and highly complex computations[1].

With the emergence of 6G which is slated to be released in the early 2030's, most of the emerging applications to accompany its release such as holographic communication and extended reality(XR) which are computationally-intensive and time sensitive will be difficult to be processed on mobile devices with limited resources[2].The key to offsetting most of the computational-intensive tasks since the release of 3G has always been with The Cloud but with the rise and mutation of wireless communications to accommodate new devices, this solution has proven to come with its own limitations such as the long distances between the end devices to the data centres has proved quite unreliable when dealing with real-time applications[3].

Edge computing is a computing paradigm that employs processing data closer to the periphery or at the edge of networks.It bridges the gap between end users and the cloud and mitigates the over-reliance on cloud infrastructure and minimizes latency.As the number of connected devices continue to soar,Multi-Access Edge computing(MEC) is integral to dynamic environments such as 6G in that it shifts computational resources to the edge of the network hence enabling mobile devices to offload their tasks to nearby edge servers(ES)[4].

In MEC the challenge of efficient task offloading is multifaceted due to the limited resources available on ES's compared to cloud servers.Determining the right task to offload is essential as overloading to an

*Corresponding author

Email addresses: leon.changara@gmail.com (Leon Changara Chemwor Odari), davies.segera@uonbi.ac.ke (Davies Rene Segera)

[1]ORCID: 0009-0006-6287-0097

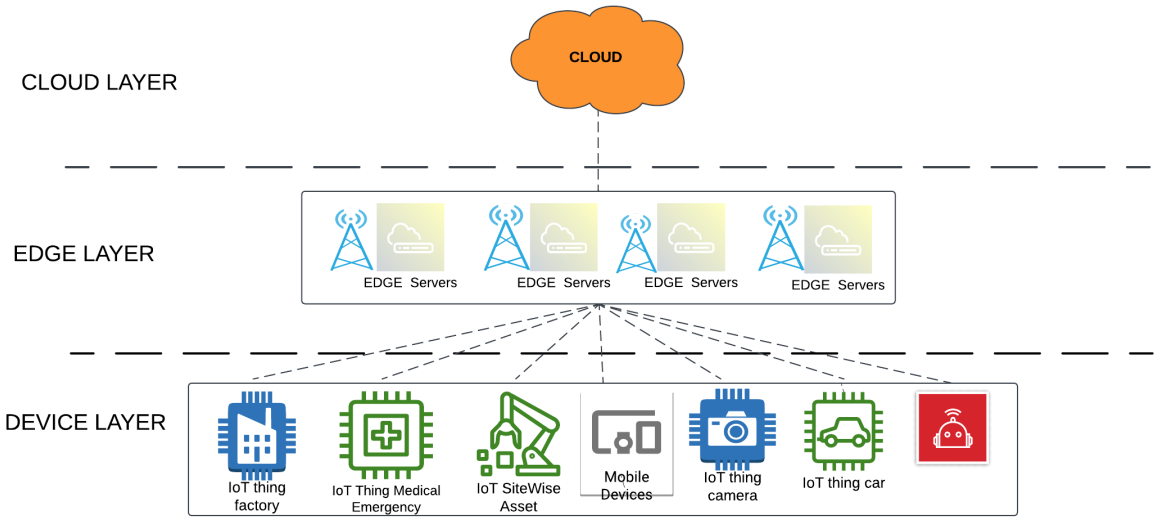


Figure 1: A Simple MEC Overview.

already overloaded ES can cause delays[5].Moreover, large data transfers between devices and servers may also lead to high transmission delays.Due to the practicality of these issues there's an need to develop an optimization algorithm that analyses the cost-benefit of offloading a task to the edge versus executing it locally.Recent studies reveal that in order to achieve optimized edge computing and task offloading algorithms, energy efficiency together with response delay are two main optimization goals one needs to consider[6]. In this paper we propose to leverage evolutionary game theory and the grey wolf optimizer to address the challenge of optimized task offloading and edge computing in a 6G environment.Our focus will be two fold one whereby we aim to optimize the offloading decisions of the mobile devices (MD's) ensuring that computationally intensive tasks are efficiently offloaded to the Edge Servers (ES's) while minimizing latency and energy consumption.Two optimizing the resource allocation on the servers side. The contributions to made by this paper are as follows :

1. A novel base approach EGT-GWO optimization algorithm targeted for task offloading in a highly dynamic 6G network.

2. An overall analysis as to why evolutionary algorithms perform better in optimization strategies

2. RELATED WORK

In this section we'll look into the offloading algorithms that have been associated with edge computing[7]. [8] studied multi-use computation offloading in a multi-channel wireless interference environment.The authors showed that it was NP-hard to compute a centralized optimal solution for edge-cloud computing hence adopted a game theory approach to offload computation in a distributed way.The algorithm achieves a Nash equilibrium which is quite static and might fail to adapt quickly in dynamic environments such as 6G.[9], proposed an offloading algorithm based on evolutionary game theory and reinforcement learning also under a multi-user environment.

[10],used particle swarm optimization (PSO) to reduce latency and dependability in MEC enabled (AR) services and analysed the trade off between latency and dependability while considering the likelihood of failure.[11],formulated a strategy called enhanced improved particle swarm optimization (EIPSO) based on the PSO algorithm to address the system delay and energy consumption associated with MEC deployment in 5G communication networks.Simulation results indicate that the algorithms outperform all the other PSO algorithms in terms of reducing total system cost, minimizing delay and balancing MEC server loads.[12],used a (GA) algorithm and a greedy strategy in a multi-user multi-server environment to reduce the cost of communication energy and delay.[13],talked of a hybrid approach algorithm based on Lyapunov optimisation and GA to reduce the overall system latency in a multi-user MEC environment on full offloading.[14],proposed an offloading algorithm based on meta-heuristic search to maximize on the throughput in a multi-user single server environment.The works in [15] utilized a three game Stackleberg game in an edge-cloud cooperative scheme considering task offloading,resource pricing optimization and content caching.This study used the three-strage equilibrium strategy algorithm based on backward induction (TEBI) algorithm to achieve Stackleberg equilibrium efficiently thereby demonstrating that cooperative game strategies could

reduce task processing delays and enhance participant utility.Modifications to the GWO optimizer has shown its effectiveness in enhancing its exploration capabilities.The use of master slave binary grey wolf optimization (MSBGWO) in [16] to solve high-dimensional biomedical datasets and outperforming other binary meta-heuristic algorithms is credible proof in using GWO-based methods to solving highly complex optimization problems.In [5],used GWO to perform a trade off between the minimization of response time and energy consumption

2.1. Key Observations

Based on the literature review, some of the inferences that can be drawn are:

1. Static algorithms that are Nash-equilibrium-based aren't fully fit to be used in highly dynamic environments such as 6G, hence the need to explore for more adaptable models that are evolutionary game theoretic combined with optimization techniques.

2. The success of EIPSO and MSBGWO showcases the effectiveness of meta-heuristics with hybrid enhancements to improve their scalability and adaptability.

3. GREY WOLF OPTIMIZER

The Grey Wolf Optimizer(GWO) algorithm is a meta-heuristic algorithm that is nature-inspired by the social hierarchy and hunting mechanism of grey wolves[17].The social hierarchy of grey wolves is in 4 different ranks of wolves in a pack:

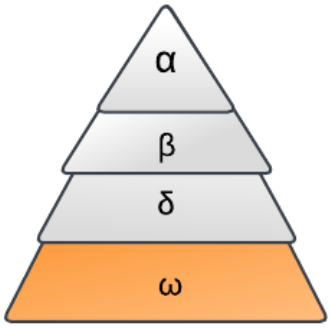


Figure 3: A diagram illustrating the socio-hierarchy of Grey Wolves.

1. Alpha wolf-This is the leader of the pack and is in charge of making decisions regarding hunting and where to hunt.

2. Beta wolf-Is the second in chain of command and assists the alpha wolf in making decisions and maintaining discipline within the pack.They are also the ones to assume leadership if the alpha wolf dies or is too old.

3. Delta wolf-Follows the alpha and the beta serving as a subordinate but are superior to the rest of the remaining wolves.

4. Omega wolf-This is the lowest ranking wolf.They are the scapegoat of the pack and are often followers and represent the general population.

The hunting strategy of the grey wolves are in the following steps:

1. Search for prey.

2. Tracking,chasing and approaching.

3. Encircling and harassing the prey until they stop moving.

4. Attacking the prey.

This strategy can be modelled mathematically using three main steps:

3.1. Encircling Phase

The positions of the wolves are updated relative to the prey.

$$\vec{\nabla} = \left| \vec{Q} \cdot \vec{G}_p - \vec{G}_w \right| \tag{1}$$

$$\vec{G}(k+1) = \vec{G}_p - \vec{F} \cdot \vec{\nabla}. \tag{2}$$

where:

$\vec{\nabla}$: distance of how far a wolf is from a leader(α, β, or δ)

$\vec{G}(k+1)$: Is the position of the new wolf

$\vec{G}_p$: Position vector of the best predator (represented by α, β, or δ).

$\vec{G}_w$: Position vector of the current wolf.

$\vec{F}$ and $\vec{Q}$:Are the coefficient vectors controlling exploration and exploitation.

The vectors $\vec{F}$ and $\vec{Q}$ are calculated as follows:

$$\vec{F} = 2 \cdot f \cdot \vec{n}_1 - f, \tag{3}$$

$$\vec{Q} = 2 \cdot \vec{n}_2, \tag{4}$$

where:

f: Linearly decreases from 2 to 0 over the course of iterations, balancing exploration and exploitation.

$\vec{n}_1$ and $\vec{n}_2$: Random vectors in the range [0, 1].

Input:Objective function
Begin
Define the search dimensions
Initialize the population of wolves with random positions in the search dimensions. G_w ,$(i = 1, 2, 3, \ldots, N)$.
Set initial counter p = 0
Calculate the fitness value of each wolf and rank them, i.e.,(G_α = best solution to the problem, G_β = second best solution to the problem, and G_δ = third best solution to the problem.)

While ($p <$ maximum number of iterations)

For each wolf G_w

Update the position of the current wolf based on the distance from the alpha,beta and delta

End for

Evaluate the fitness of all wolves

Update the positions of G_α, G_β and G_δ based on new fitness values

Update iteration counter p = p + 1

End while

Output : G_α as the best solution obtained

End

Figure 2: GWO Pseudocode.

3.2. Hunting Phase

The hunting phase is guided by three best wolves: alpha, beta and delta. The equations are given by:

$$\nabla_\alpha = |Q_1 \cdot G_\alpha - G|, \tag{5}$$

$$\nabla_\beta = \left|Q_2 \cdot G_\beta - G\right|, \tag{6}$$

$$\nabla_\delta = |Q_3 \cdot G_\delta - G|, \tag{7}$$

$$\vec{G_1} = \vec{G_\alpha} - \vec{F_1} \cdot \vec{\nabla}_\alpha, \tag{8}$$

$$\vec{G_2} = \vec{G_\beta} - \vec{F_2} \cdot \vec{\nabla}_\beta, \tag{9}$$

$$\vec{G_3} = \vec{G_\delta} - \vec{F_3} \cdot \vec{\nabla}_\delta. \tag{10}$$

Finally,the remaining wolves update their positions based on the positions of these three leaders.

$$G_{k+1} = \frac{\vec{G_1} + \vec{G_2} + \vec{G_3}}{3} \tag{11}$$

3.3. Attacking(Exploitation Phase)

As the wolves approach the prey, the value of the the $\vec{F}$ decreases leading the wolves to focus more on exploitation(local search) rather than exploitation(global search).When $\vec{F}$ reaches zero, the algorithm converges to the optimal solution.

4. EVOLUTIONARY GAME THEORY

Game theory is a mathematical theory of socio-economic phenomena exhibiting interactions among decision makers(agents),whose actions affect each other.What you need to understand before we delve deep is that the word "game" in game theory refers to any interaction between multiple players in which each players' pay-off is affected by the decisions of others.Classical game theory (CGT) is primarily focused on rational decision making in strategic scenarios.The players are assumed to be rational and fully informed while aiming to capitalize on strategies that maximize their payoffs in a given scenario.CGT is often in a static environment and the decisions made by players are are based on fixed rules and payoffs. Evolutionary Game Theory (EGT) analyzes how strategic decisions evolve when multiple players interact, it focuses mainly on the dynamic adaptation,identifying optimal strategies that maximize success in competitive environments through iterative learning and adjustment,[18].In contrast to CGT,EGT dwells in a dynamic environment.For this paper we plan to utilize Logit Dynamics strategy (LD).This strategy has its base foundation under evolutionary game theory as it focuses on situations where individuals choose their strategies probabilistically, influenced by expected payoffs.

4.1. Logit Dynamics

We can represent the probability of selecting a strategy mathematically using the softmax function as:

$$\Upsilon_i = \frac{e^{\beta \Xi_i}}{\sum_{m=1}^{n} e^{\beta \Xi_m}} \tag{12}$$

Υ_i represent the probability of selecting a strategy i while Ξ_i represents the payoff of strategy i. A temperature parameter β is used to adjust the randomness in

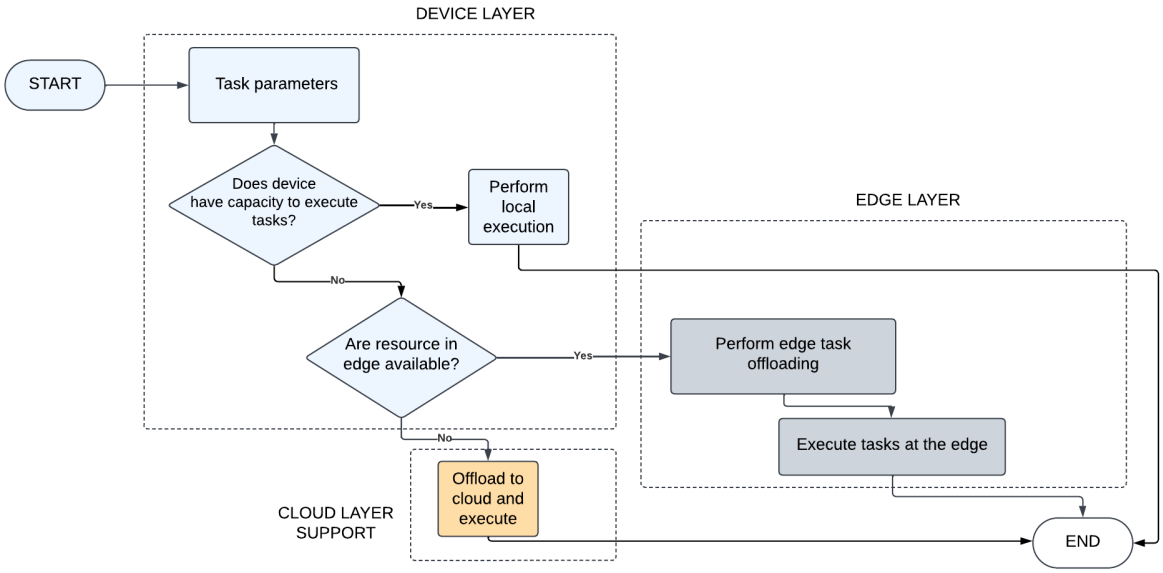


Figure 4: Task offloading and edge compute flow diagram.

decision making.This parameter can only be investigated as it approaches infinity or zero.A high temperature favours more exploration as the exponential term becomes less prone to randomness almost all strategies are equally likely to be selected.A low temperature (as it approaches zero) favours more exploitation as the strategy with the highest payoff is more likely to be selected.This parameter will become inherently useful in the beginning and later stages of our optimization process.The denominator is a normalization term that ensures all probabilities add up to one.

5. METHODOLOGY

5.1. System Model and Overview

We consider our model in three sections as illustrated in figure(1) based on a 6G environment which includes ESs, MDs and CS as a back up option.We let MD $= \{\mathrm{MD}_1, \mathrm{MD}_2, \ldots, \mathrm{MD}_n\}$ denote the set of devices,they are are considered as a general term for all the devices in the device layer including Iot devices.Each of them are resource constrained and they have a computationally intensive task that need to be done.The tasks are denoted by T_i using a tuple $(\mathrm{Ts}_i, \mathrm{C}_{\mathrm{load}}, \tau_i)$ where Ts_i is the task size.C_{load} is the concurrent computational demand (CPU cycles in bits) and τ_i is the latency sensitivity parameter which represents the maximum allowable time for a task to be completed within real time.We also consider a number of n edge servers denoted by $\Gamma = \{\Gamma_1, \ldots, \Gamma_n\}$.The edge servers are in constant communication with MDs to know the number of devices connected.

5.2. Problem Formulation

We need to analyze task offloading from two fronts, that is, at the local level(MD layer) and the ES layer. The execution of tasks in the MD depends on the computational capability of the device itself and the energy consumption used. The execution of a set of tasks Ts_i on an MD can be expressed as:

$$\Delta_{lt} = \frac{\mathrm{C}_{\mathrm{load}}}{v_{m_d}} \tag{13}$$

and the subsequent energy consumed is given by:

$$\mathcal{E}_L = \mathrm{C}_{load}(v_{m_d})^2 k. \tag{14}$$

where C_{load} is the computational demand in cycles/bit and K is the device specific energy coefficient.

For computation to take place in the ES, the execution time is characterized by both the transmission delay θ_d and the execution time Δ_{et} at the edge server, expressed as:

$$\theta_d = \frac{\mathrm{ts}_i}{S_{cc}}, \tag{15}$$

$$\Delta_{et} = \frac{\mathrm{C}_{\mathrm{load}}}{v_{\mathrm{ES}}} \tag{16}$$

where ts_i is the task size in bits and S_{cc} represents the Shannon channel capacity formula. Hence, the total execution time Δ_E at the edge server can be given by:

$$\Delta_E = \theta_d + \Delta_{et} \tag{17}$$

The energy consumed to offload a task to the ES $\mathcal{E}_{\mathrm{off}}$ consists of:

$$\mathcal{E}_{\mathrm{off}} = \mathcal{E}_\Gamma + \mathcal{E}_{\mathrm{trans}} \tag{18}$$

where $\mathcal{E}_\Gamma$ is the energy consumed at the edge server and $\mathcal{E}_{\mathrm{trans}}$ is the transmission energy

5.2.1. Task costs

A task is offloaded if the following decision criteria holds to be true:

$$\Delta_{lt} > \Delta_{et} \tag{19}$$

and

$$\mathcal{E}_L > \mathcal{E}_{\text{off}} \tag{20}$$

On the assumption that a task is offloaded onto the cloud if the ES resources are unavailable. Our aim is to obtain optimality in edge computing and task offloading in dynamic 6G networks,in order for this to be achieved we need to consider the task costs and penalties for overloading the ESs. The task costs account for latency, energy and the weights associated;

Table 1: Summary of Key Notations

Notation	Description
Md	Set of mobile devices in the system.
T_i	A task represented as a tuple.
ts_i	Task size in bits.
C_{load}	Computational demand of a task in CPU cycles per bit.
τ_i	maximum allowable time for a task to be executed.
Γ_n	Set of edge servers,
Δ_{lt}	Local execution time of tasks on MDs,
$\mathcal{E}_L$	Energy consumption for local execution of tasks,
v_{m_d}	processing speed of the MD
θ_d	Transmission delay for task offloading,
Scc	Represents the Shannon Capacity Formula
Δ_{et}	Execution time at the edge server
Δ_E	Total execution time at the edge server
$\mathcal{E}_{\text{off}}$	Energy consumed for task offloading
ω_1	Weights associated with the cost of offloading to the ES
ω_2	Weights associated with local execution
$\mathcal{E}_\Gamma$	energy consumed by a task at the edge

Latency (Δ_i): Total execution time (local or offloaded) for a task T_i, can be calculated as:

$$\Delta_i = \begin{cases} \Delta_{lt} & \text{(local_execution)} \\ \Delta_E + \theta_d & \text{(offloaded)} \end{cases}$$

Energy (E_i): Total energy consumed, given by:

$$\mathcal{E}_i = \begin{cases} \mathcal{E}_L & \text{(local execution)} \\ \mathcal{E}_{\text{off}}, & \text{(offloaded)} \end{cases}$$

The weights ω_1 and ω_2 are associated to time and energy respectively and they govern their importance.

5.2.2. Overloading Penalty

When a servers load exceeds its available capacity,it can't handle tasks efficiently and this leads to increased latency and task execution failures.Therefore it is necessary to include a penalty term to limit overloads.The overloading penalty on edge servers is needed to ensure that multiple tasks aren't offloaded to the same server.The penalty can be; the current load on the server L_Γ,the computational capability C_Γ of the server and the penalty coefficient P_k that determines the severity of overloading on an edge server.The penalty term can be expressed mathematically as:

$$P_k \cdot \max(0, L_\Gamma - C_\Gamma) \tag{21}$$

only when

$$L_\Gamma > C_\Gamma \tag{22}$$

5.2.3. Objective Function

We can therefore derive an objective function J based on the above decisions as:

$$\begin{aligned} J_T = \min\Bigg(& \sum_{i=1}^{m} [\omega_1 \cdot \Delta_i + \omega_2 \cdot \mathcal{E}_i] \\ & + \min \sum_{k=1}^{m} (P_k \cdot \max(0, L_\Gamma - C_\Gamma)) \Bigg) \end{aligned} \tag{23}$$

*5.3. **Proposed Algorithm***

5.3.1. Task offloading decision

The first approach is task offloading with the use of LD, the mobile devices probabilistically determine whether to offload tasks to the ES or process them locally.This decision depends on the expected payoff for each strategy(offload or local).The payoff for offloading a particular task to the ES is determined by a combination of latency and energy in equation (12). The general idea behind this is that the lower the latency and energy consumption,the higher the payoff for that strategy(offloading to that particular ES).The latency and energy associated with offloading a task to an edge server is given by equation (17) and equation(18).

5.3.2. Pay-off calculation

The payoff for offloading a task to an ES should be inversely proportional to the total cost;remember if we aim to minimize both energy and latency a lower cost(sum of energy and latency) results in a higher pay-off.Pay-off for offloading a task *"i"* to an edge server *"j"*:

$$\Xi_{ij} = -(\omega_1 \cdot \Delta_{ij} + \omega_2 \cdot \mathcal{E}_{ij}) \tag{24}$$

If both latency Δ_{ij} and energy $\mathcal{E}_{ij}$ are low for a particular edge server j the pay-off,U_{ij} will be high(less negative).Intuitively,if the latency and energy consumption

are high for a particular server,the pay-off,U_{ij} will be low(more negative).The system will thus use equation (12) to determine the probability of offloading the task to the server with the highest pay-off. For local processing,the pay-off calculation follows a similar logic but focuses on the latency and energy consumption associated with processing the task locally on the MD:

$$\Xi_i^{local} = -(\omega_1 \cdot \Delta_i^{local} + \omega_2 \cdot \mathcal{E}_i^{local}) \quad (25)$$

The system will compare the payoff U_{ij} for offloading the task to an ES with the payoff for local execution Ξ_i^{local}.Local processing will take place only when:

$$\Xi_i^{local} > \Xi_{ij} \iff ((\omega_1 \cdot \Delta_{\text{lt}} + \omega_2 \cdot \mathcal{E}_L) < (\omega_1 \cdot \Delta_E + \omega_2 \cdot \mathcal{E}_{\text{off}})) \quad (26)$$

Task offloading to the ES will take place when;

$$\Xi_i^{local} < \Xi_{ij} \iff ((\omega_1 \cdot \Delta_{\text{lt}} + \omega_2 \cdot \mathcal{E}_L) > (\omega_1 \cdot \Delta_E + \omega_2 \cdot \mathcal{E}_{\text{off}})) \quad (27)$$



Figure 5: Task offloading decision process.

5.3.3. Resource allocation

Grey wolves represent different resource allocation strategies for the edge servers in the 6G network.Each wolf's position corresponds to a specific allocation of resources at the edge servers and the algorithm iterates through various positions to find the optimal allocation that minimizes the objective function.

5.3.4. GWO fitness function

The fitness function is used to evaluate how well each potential resource allocation meets the optimization objectives;minimizing latency,minimizing energy consumption and load balancing (ensuring edge servers are not overloaded).The function can be represented as:

$$f(x) = \sum_{i=1}^{N} ((\omega_1 \cdot \Delta_E(x) + \omega_2 \cdot \mathcal{E}_{\text{off}}(x)) + P_k \quad (28)$$

x: represents a specific allocation of resources across edge servers.

$\Delta_E(x)$: is the latency of the task based on the current resource allocation x

$\mathcal{E}_{\text{off}}(x)$: is the energy consumption of the task based on the current resource allocation x

P_k: is the penalty term

5.3.5. Position update in GWO

Wolves are ranked based on their fitness value, with the best wolf(alpha wolf) being the one with the lowest cost and best resource allocation.Wolves update their positions based on the three best wolves (α,β,δ) at the pecking order using the equation (11)

5.3.6. GWO optimization loop

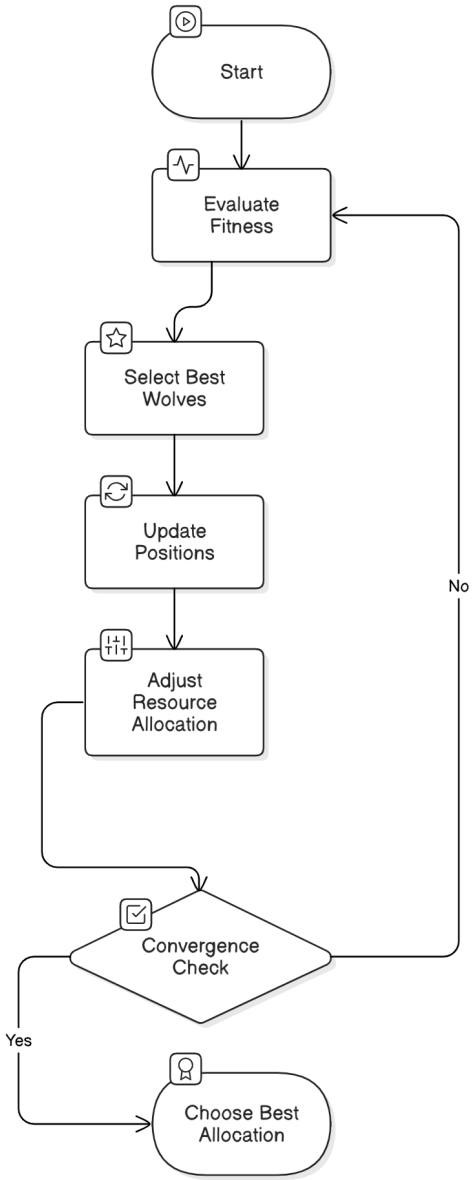


Figure 6: GWO optimization loop.

Input: Sets of tasks T, edge servers Γ, mobile devices MD, weights ω_1, ω_2,Logit parameter β
Begin
Initialize strategy probability matrix Υ for task offloading decisions
Define search dimensions as number of tasks n
Perform Logit Dynamics iterations to reach equilibrium for offloading strategies
Initialize GWO population with half based on Logit probabilities and half random
Calculate initial fitness using hybrid payoff function that considers:

- Latency and energy for both local and edge processing
- Resource constraints and load balancing

Rank solutions: G_α (best), G_β (second), G_δ (third)
Set iteration counter $p = 0$

While ($p < Iterations$)
Update Logit dynamics strategies based on current resource allocation
Calculate convergence coefficient $f = 2 - 2 \times \frac{p}{Max_iter}$
For each wolf G_w
Generate random coefficients for position updating
Calculate distance vectors to alpha, beta, and delta wolves (∇_α, ∇_β, ∇_δ)
Update position based on weighted influence of alpha, beta, and delta using the *equation* (11)
Ensure positions are valid allocation options (0 to m)
End for
Evaluate fitness of all wolves with updated positions
Update alpha, beta, and delta wolf rankings
Update iteration counter $p = p + 1$
End while
Output: G_α as the best task allocation solution

End

Figure 7: LD-GWO Pseudocode.

The iteration continues until the system converges to the best possible resource allocation.

5.3.7. Iterative feedback loop

LD determines the offloading decision(task assignments) while GWO optimizes the resource allocations at the edge servers.The results from both processes are combined in the next iteration to refine the decisions.The loop continues until convergence.

5.4. Simulation Environment and Experimental Setup

All simulations were performed using Python installed locally on a HP-840 with an intel(R) i5-6300U CPU @ 2.40 GHz,8GB RAM,256GB SSD storage running on 64-bit Windows 11 operating system.Streamlit was used to help with live visualization of the simulations due to its ability to combine code,visualizations and interactive elements in a dashboard friendly interface.

5.5. Simulation environment and data generation

In order to evaluate the proposed LD-GWO algorithm under controlled conditions.The following default parameters were used to generate the dataset.

Parameter	**Value**
Number of tasks	50
Number of ESs	10
Task CPU requirements	1–5 GHz
Task Size	10–100 MBs
Task deadline	10–50 ms
Node Capacity	20–40 GHz
Bandwidth	10 GBps
Max iterations	100
Population size	30
Energy and latency weights	0.50, 0.20

Table 1: System Parameters

The dataset simulates a mobile edge computing environment with MDs and ESs whereby each task

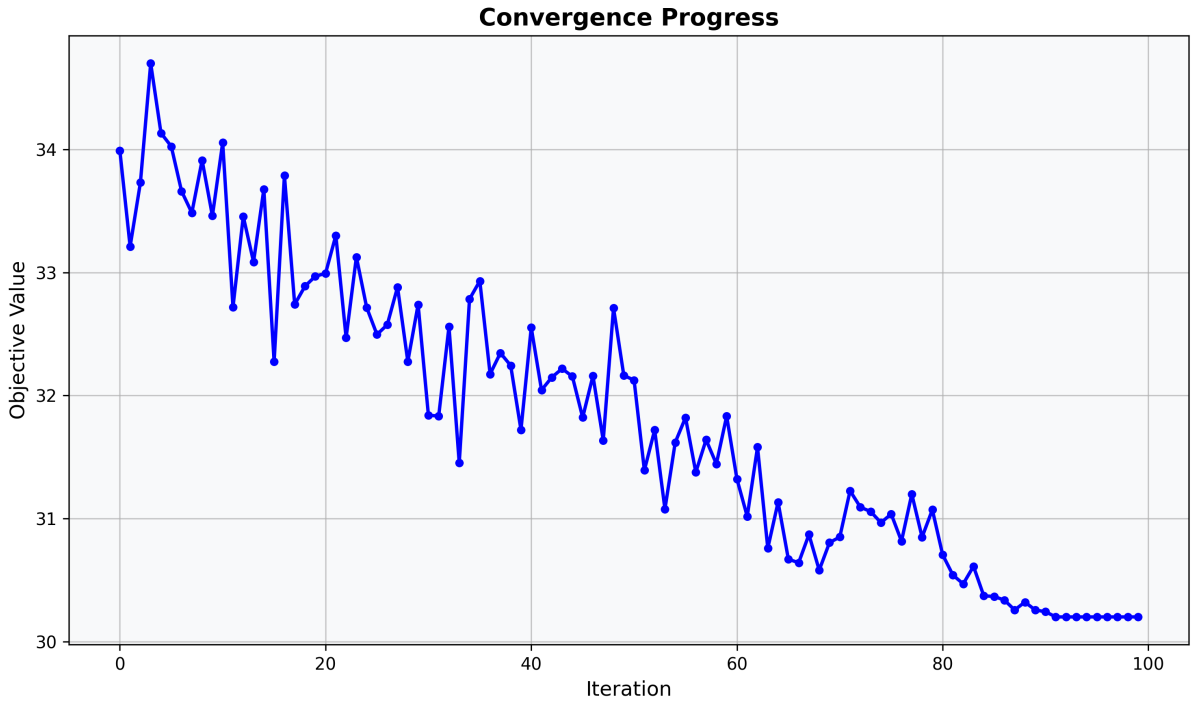


Figure 8: Convergence curve of the proposed algorithm.

is characterized by its computational demand, data size and the maximum allowable time for task execution.Each edge node was characterized by its processing capability, power consumption and its location on a 100 by 100 grid size.A fixed bandwidth was used to model the communication links between the tasks and edge nodes.

6. RESULTS AND ANALYSIS

To comprehensively evaluate the proposed algorithm we performed two distinct simulation scenarios due to limitations of hardware capability and complexity of the live algorithm implementation:

6.1. Scenario 1:Standalone analysis of the proposed algorithm:

In this scenario the primary goal was to investigate the intrinsic behaviour and performance characteristics of the algorithm itself without comparing it with other algorithms.We analyzed its convergence properties in order to understand and analyse how the algorithm was obtains optimal solutions based off the objective function, its sensitivity to internal parameters such as the specific weights within the logit model and its performance under varying loads(such as number of tasks,edge nodes and number of iterations).

6.1.1. Convergence analysis

The convergence curve of the proposed algorithm shown in figure 8 show its trajectory over 100 iterations.During the initial 0-20 iterations the algorithm is still exploring the solution space to identify promising regions,this can be attributed to the grey wolves robust searching ability as there are alot of rapid movements.After the 20th iteration the we can observe that there is some gradual refinement of the objective value.This can be attributed to the Logit models capability in fine tuning the solutions through the offloading decision making process.The reduction in the number of oscillations after 70 iterations suggest a stable performance with the algorithm being bale to mitigate local optima convergence.

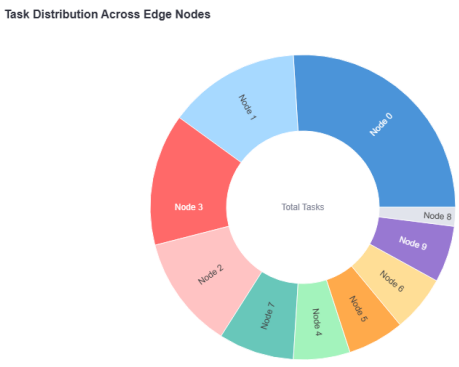


Figure 10: Sunburst Chart showing Task distribution across Nodes.

6.1.2. Node-Resource utilization

Each node across the simulation environment has different computational capacities.Evaluation of the tasks across 10 of this nodes as shown in figure 10

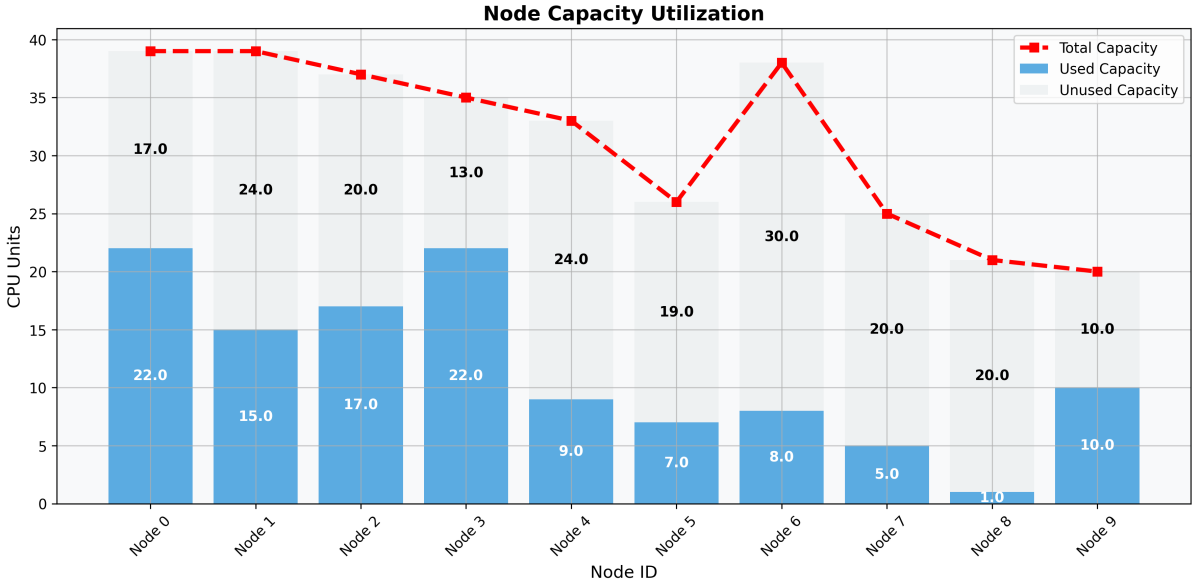


Figure 9: Node utilization across edge servers.

Table 2: Complete Performance Summary of Algorithms (Values Rounded to 4 Decimal Places)

Algorithm	Fitness	Energy	Response Time	QoS Diff.	Resource Util.	Energy Aware Compl. Efficiency
LD-GWO	0.6105	0.5378	0.4053	0.5106	0.4454	0.1540
RD-GWO	0.5374	0.5407	0.4116	0.5006	0.4438	0.1243
FP-GWO	0.5412	0.4751	0.4114	0.4896	0.4474	0.1195
BRD-GWO	0.5328	0.4786	0.4291	0.4673	0.4538	0.0890
WOA	0.1479	0.5200	0.7777	0.5112	0.4716	0.1647
HHO	0.3120	0.5380	0.5759	0.4831	0.4543	0.1191
SSA	0.2965	0.5447	0.6164	0.4829	0.4428	0.1220
AO	0.2346	0.4966	0.6647	0.5079	0.4817	0.1589
RSA	0.3424	0.5085	0.5691	0.4587	0.4725	0.1214
TSA	0.2631	0.5008	0.6706	0.4742	0.5054	0.1814
COA	0.2525	0.5214	0.6664	0.4854	0.4509	0.1508
AVO	0.2105	0.5205	0.6969	0.4701	0.5050	0.1736
DO	0.2544	0.5030	0.6584	0.5056	0.4797	0.1093
GTO	0.1971	0.5363	0.6923	0.5036	0.4781	0.1272
SHO	0.1669	0.5170	0.7460	0.5052	0.4500	0.1823

indicate that the algorithm offloads the most number of tasks to the the edge nodes that have the highest computational capacity in this case Node 0 and Node 1.This results solidifies the algorithms ability to prevent congestion and overloading of tasks in one edge server by keeping offloaded tasks under the computational threshold provided.

6.2. Scenario 2:Comparative Performance Evaluation:

In this scenario we compared our proposed algorithm (LD-GWO) against other relevant algorithms that were selected to represent different classes of offloading strategies and optimisation techniques commonly used in edge computing and task offloading.Given that our LD-GWO is a hybrid metaheuristic algorithm, comparison against other metaheuristics is key in showing how it performs relative to them.

Meta-heuristic algorithms selected for this comparison where HHO (Harris Hawks Optimization) [19], SSA (Salp Swarm Algorithm) [20], AO (Aquila Optimization) [21], RSA (Reptile Search Algorithm) [22], TSA (Tunicate Swarm Algorithm) [23], AVO (African Vultures Optimization) [24], WOA (Whale Optimization Algorithm) [25], GTO (Gorilla Troops Optimization) [26], SHO (Sea Horse Optimizer) [27], COA (Coati Optimization Algorithm) [28], and DO (Dandelion Optimizer) [29]..For comparison against evolutionary game theoretic strategies, we employed four unique strategies based off evolutionary game theory to the grey wolf optimizer to come up with 3 comparable algorithms i.e the BRD-GWO(Best Response Dynamics)[30],FP-GWO(Fictitious Play)[31] and RD-GWO(Replicator Dynamics)[32].These 3 algorithms use the standard GWO optimizer together

Table 3: Statistical Comparison of Algorithms (Values Rounded to 4 Decimal Places)

Comparison	p-value	Statistic	Median Diff	Percent Diff	Corrected p-value	Significant
LD-GWO vs WOA	0.0000	0	0.4765	341.6619	0.0000	true
LD-GWO vs SSA	0.0000	0	0.3053	98.3037	0.0000	true
LD-GWO vs AO	0.0000	0	0.3918	174.7648	0.0000	true
LD-GWO vs HHO	0.0000	1	0.3085	100.3426	0.0000	true
LD-GWO vs RSA	0.0000	6	0.2743	80.2837	0.0000	true
LD-GWO vs BRD-GWO	0.0000	19	0.0802	14.9761	0.0000	true
LD-GWO vs RD-GWO	0.0000	24	0.0717	13.1744	0.0000	true
LD-GWO vs FP-GWO	0.0000	35	0.0903	17.1831	0.0000	true
FP-GWO vs BRD-GWO	0.5978	206	−0.0101	−1.8834	0.6576	false
RD-GWO vs BRD-GWO	0.5978	206	0.0085	1.5920	0.6576	false
RD-GWO vs FP-GWO	0.8078	220	0.0186	3.5420	0.8078	false

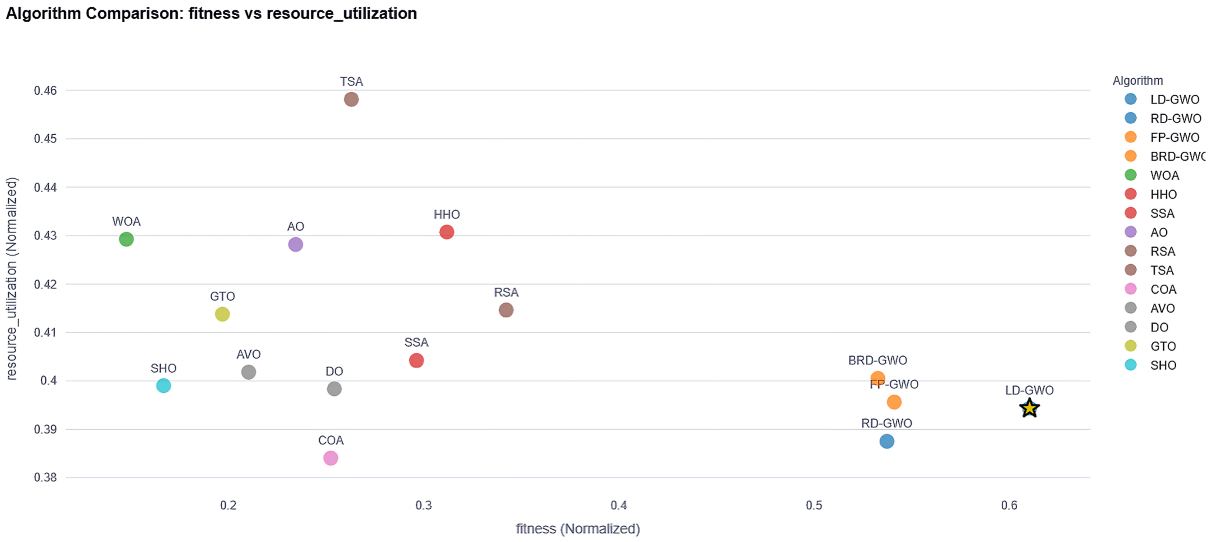


Figure 11: Fitness vs Resource utilization Plot



Figure 12: Fitness vs Qos differentiation Plot

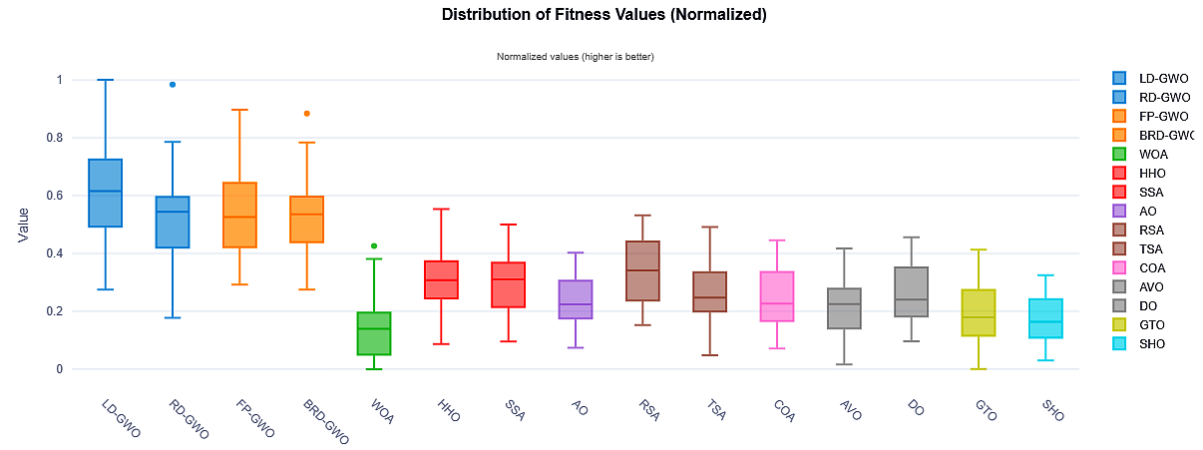


Figure 13: Distribution of fitness across algorithms

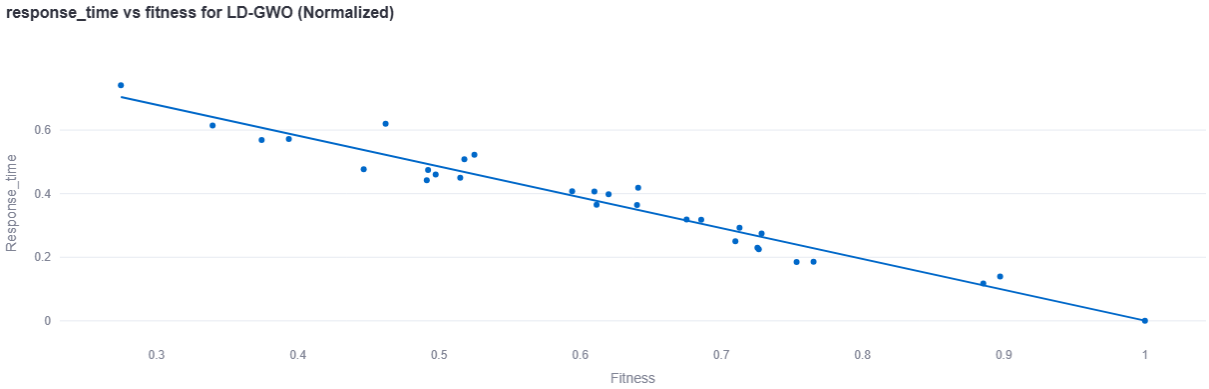


Figure 14: Response vs Fitness plot

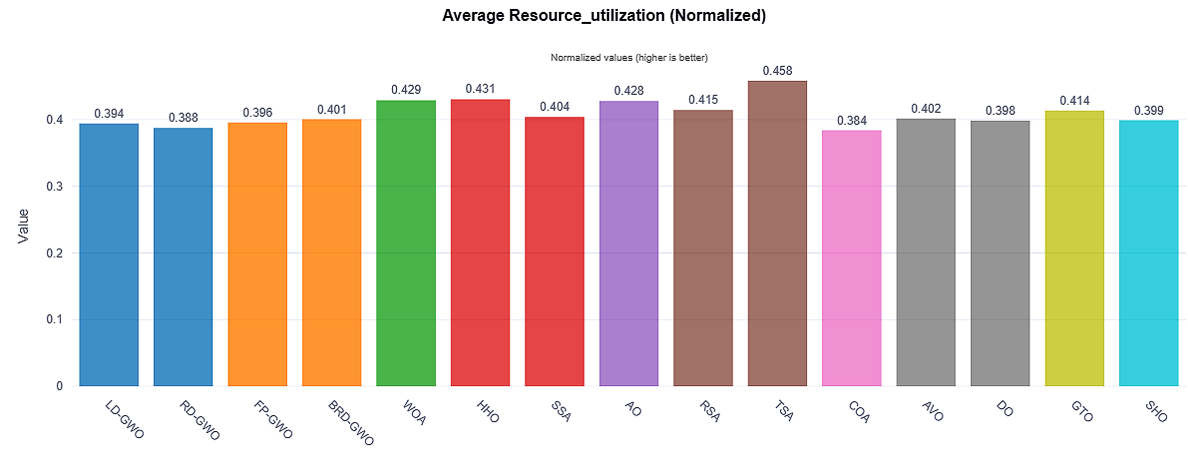


Figure 15: Resource utilization Plot

with their corresponding evolutionary game theory strategy for optimization. In the table 2 we show the performance summary of the 15 algorithms across different performance metrics after undertaking a total of 30 trials then the result is averaged.

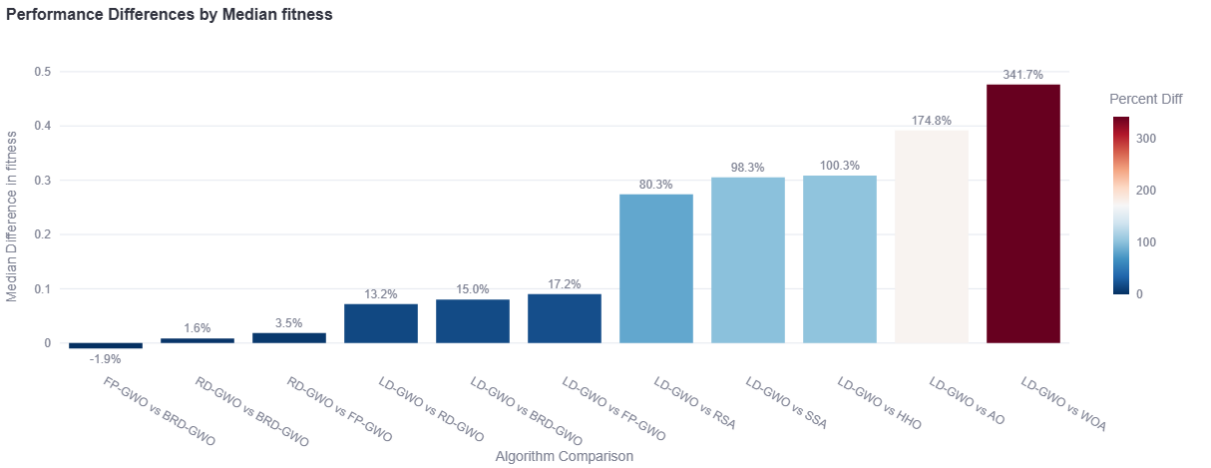


Figure 16: Statistical Performance Difference by Median fitness

6.2.1. Fitness

The fitness metric is associated to the quality of the optimization function.LD-GWO achieves highest mean fitness of 0.6105 surpassing other game theoretic GWO variants, RD-GWO(0.5374),FP-GWO(0.5412) and BRD-GWO(0.5328). Other meta-heuristics exhibited even lower values with WOA having the lowest fitness.This is an indication that LD-GWO achieves a balanced trade off between latency and energy consumption.

6.2.2. Fitness vs Resource allocation

In the graph 11,which is a scatter plot that shows the placements of the 15 algorithms when fitness is analyzed against resource utilization.LD-GWO is highlighted as a star symbol on the graph.LD-GWO has a resource efficiency just under 0.4 which places it in the the lower half of the resource index.Most algorithms show a utilization of between 0.4 and 0.44 but they posses a lower fitness.LD-GWO manages to strike a balance between fitness and resource utilization.

6.2.3. Fitness vs Response time

The graph 14 shows the correlation between fitness and response time.The trend is a negative slope indicating an inverse relationship.As the fitness increases, the response time decreases.This is a desirable result as it indicates that higher values are correlated to faster response times.LD-GWO has the highest fitness hence it becomes more efficient in terms of response time as it achieves better solutions.

6.2.4. Fitness vs QOS differentiation

QOS(Quality of service) is the ability of a system or algorithm to handling varying levels of tasks efficiently based on the available resource.In dynamic 6G network tasks such as telemedicine which are mission-critical tasks would require first hand prioritization against other less critical tasks.The scatter plot in 12 shows LD-GWO together with the other game theoretic variants as having a better QOS differentiation than the other meta-heuristics.In this metric comparison, LD-GWO is in the 90th percentile with a QOS differentiation of about 0.47.Most other algorithms are clustered between 0.4 and 0.44.

6.3. Statistical Analysis

Friedman test together with a Wilcoxon signed rank test was performed on the algorithms.The Friedman test yielded a statistic of 76.12 with a p value of 0.000 between the GWO game-theoretic variants.This is an indication that atleast one variant(LD-GWO) outperforms the others in its subclass.The Wilcoxon signed rank test as shown in table 3 was performed between the subclass of GWO game-theoretic variants and a few of the other meta-heuristic algorithms. Pairwise comparisons between the proposed algorithm and other meta-heuristics exhibited very low p-values with very high percentage differences as shown in the graph 16 and in table 3.All LD-GWO comparisons in this group of comparisons remain significant after correction, strengthening evidence and cementing confidence in the proposed algorithms performance metrics.Comparisons between the GWO variants themselves (LD-GWO vs RD-GWO,BRD-GWO and FP-GWO) show relevantly higher p-values ¿ 0.5 indicating no significant differences among them.The relatively low median differences and percentage differences further support this claim.

7. Conclusion

This paper aimed to address the problem of optimization in task offloading and edge computing in 6G networks with the main objective of reducing energy consumption,latency minimization and efficient task resource handling in edge servers.The paper implemented the contribution of a novel evo-

lutionary game theoretic grey wolf optimizer algorithm design to tackle the problem.The performance of the proposed algorithm was analysed and its fitness showed significant performance in the overall optimization of the objective function and also being able to handle the latency and energy trade offs effectively.Comparative analysis done in scenario 2 showed that there were significant performance differences between LD-GWO and non-GWO variants based on the objective function.The research on this paper highlights the efficiency of of using game theoretical frameworks and principles together with metaheuristics to further refine optimization techniques. The major constraints encountered during the research included the complexity in implementing the algorithm in a live simulation and also hardware limitations that resulted in the algorithm taking too long too converge when comparing against other optimization techniques.Further areas of improvement would include modelling the algorithm to be able to handle the scalability aspect in 6G environments and real-world constraints associated with the network.

8. Appendix : Code Repository

The code implementation,datasets and additional documentation can be found at: `https://github.com/jaGaban747/EGT-GWO-Project-Final` and `https://github.com/jaGaban747/EGT-GWO`.

References

[1] Hui Huang, Qiang Ye, and Yitong Zhou. 6g-empowered offloading for realtime applications in multi-access edge computing. *IEEE Transactions on Network Science and Engineering*, 10:1311–1325, 2023.

[2] Ziduan Wang, Tiankui Zhang, Yuanwei Liu, and Wenjun Xu. Caching placement and resource allocation for ar application in uav noma networks. In *GLOBECOM 2020-2020 IEEE Global Communications Conference*, pages 1–6. IEEE, 2020.

[3] Firdose Saeik, Marios Avgeris, Dimitrios Spatharakis, Nina Santi, Dimitrios Dechouniotis, John Violos, Aris Leivadeas, Nikolaos Athanasopoulos, Nathalie Mitton, and Symeon Papavassiliou. Task offloading in edge and cloud computing: A survey on mathematical, artificial intelligence and control theory solutions. *Computer Networks*, 195:108177, 2021.

[4] Binbin Su, Hongbo Liu, and Jin Meng. Joint time and power allocation for wireless powered noma-mec systems. In *2021 13th international conference on communication software and networks (ICCSN)*, pages 24–28. IEEE, 2021.

[5] Aamir Abbas, Ali Raza, Farhan Aadil, and Muazzam Maqsood. Meta-heuristic-based offloading task optimization in mobile edge computing. *International Journal of Distributed Sensor Networks*, 17(6):15501477211023021, 2021.

[6] Shuchen Zhou and Waqas Jadoon. The partial computation offloading strategy based on game theory for multi-user in mobile edge computing environment. *Computer Networks*, 178:107334, 2020.

[7] Abednego Acheampong, Yiwen Zhang, Xiaolong Xu, and Daniel Appiah Kumah. A review of the current task offloading algorithms, strategies and approach in edge computing systems. *Computer Modeling in Engineering & Sciences, 134 (1)*, pages 35–88, 2023.

[8] Suzhi Wang, Ming Liu, Jie Xu, et al. Meta-offloading: Learning to adapt computation offloading in dynamic mobile edge environments. In *ACM MobiHoc*, pages 31–40. ACM, 2022.

[9] Yuya Cui, Degan Zhang, Ting Zhang, Lu Chen, Mingjie Piao, and Haoli Zhu. Novel method of mobile edge computation offloading based on evolutionary game strategy for iot devices. *AEU-International Journal of Electronics and Communications*, 118:153134, 2020.

[10] Jianhui Liu and Qi Zhang. Code-partitioning offloading schemes in mobile edge computing for augmented reality. *Ieee Access*, 7:11222–11236, 2019.

[11] Shun Li, Haibo Ge, Xutao Chen, Linhuan Liu, Haiwen Gong, and Rui Tang. Computation offloading strategy for improved particle swarm optimization in mobile edge computing. In *2021 IEEE 6th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*, pages 375–381. IEEE, 2021.

[12] Li Kuang, Tao Gong, Shuyin OuYang, Honghao Gao, and Shuiguang Deng. Offloading decision methods for multiple users with structured tasks in edge computing for smart cities. *Future Generation Computer Systems*, 105:717–729, 2020.

[13] Ni Zhang, Songtao Guo, Yifan Dong, and Defang Liu. Joint task offloading and data caching in mobile edge computing networks. *Computer Networks*, 182:107446, 2020.

[14] Yufan Xu, Yan Wang, and Junyao Yang. Meta-heuristic search based model for task offloading and time allocation in mobile edge computing. In *Proceedings of the 2020 6th international conference on computing and artificial intelligence*, pages 117–121, 2020.

[15] Lixue Gao, Xin Chen, Bo Yin, Lin Cui, and Hua Xing. Game theory based task offloading, content caching and resource pricing under edge-cloud collaboration in 6g network. In *2023 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6. IEEE, 2023.

[16] Enock Momanyi and Davies Segera. A master-slave binary grey wolf optimizer for optimal feature selection in biomedical data classification. *BioMed Research International*, 2021(1):5556941, 2021.

[17] Laura Rodriguez, Oscar Castillo, and Jose Soria. Grey wolf optimizer: A survey of variants and applications. *Neurocomputing*, 456:61–80, 2021.

[18] On Park, Hyo-Sang Shin, and Antonious Thourdos. Evolutionary game theory based multi-objective optimization for control allocation of over-actuated system. *IFAC-PapersOnLine*, 52(12):310–315, 2019.

[19] Ali Asghar Heidari, Seyedali Mirjalili, Hossam Faris, Ibrahim Aljarah, Majdi Mafarja, and Huiling Chen. Harris hawks optimization: Algorithm and applications. *Future Generation Computer Systems*, 97:849–872, 2021.

[20] Yong Zhang, Zhigang Jin, and Seyedali Mirjalili. Enhanced salp swarm algorithm with dynamic opposition-based learning for global optimization. *Expert Systems with Applications*, 207:117961, 2022.

[21] Benyamin Abdollahzadeh, Farhad Soleimanian Gharehchopogh, and Seyedali Mirjalili. Efficient optimization of real-world problems using aquila optimizer. *Neural Computing and Applications*, 34:17433–17456, 2022.

[22] Laith Abualigah, Ali Diabat, Seyedali Mirjalili, Mohamed Abd Elaziz, and Amir H Gandomi. Reptile search algorithm (rsa): A nature-inspired meta-heuristic optimizer. *Expert Systems with Applications*, 191:116158, 2022.

[23] Sandeep Kaur, Lalit Kumar Awasthi, AL Sangal, and Gaurav Dhiman. Tunicate swarm algorithm: A new bio-inspired based metaheuristic paradigm for global optimization. *Engineering Applications of Artificial Intelligence*, 90:103541, 2020.

[24] Benyamin Abdollahzadeh, Farhad Soleimanian Gharehchopogh, and Seyedali Mirjalili. African vultures optimization algorithm: A new nature-inspired metaheuristic algorithm for global optimization problems. *Computers & Industrial Engineering*, 158:107408, 2021.

[25] Juan Tu, Huiling Chen, Jiawei Liu, Ali Asghar Heidari, and Xuelong Zhang. Improved whale optimization algorithm

for solving constrained optimization problems. *Knowledge-Based Systems*, 260:110166, 2023.

[26] Benyamin Abdollahzadeh, Farhad Soleimanian Gharehchopogh, and Seyedali Mirjalili. Gorilla troops optimizer: A new nature-inspired metaheuristic algorithm for global optimization problems. *International Journal of Intelligent Systems*, 36(10):5887–5958, 2021.

[27] Wenjing Zhao, Ling Wang, and Zhenxing Zhang. Sea-horse optimizer: A novel nature-inspired meta-heuristic for global optimization problems. *Applied Intelligence*, 52:11833–11860, 2022.

[28] Amin Asadi Dehkordi, Ali Safaa Sadiq, Seyedali Mirjalili, and Farhad Soleimanian Gharehchopogh. Coati optimization algorithm: A new bio-inspired metaheuristic algorithm for solving optimization problems. *Knowledge-Based Systems*, 259:110011, 2023.

[29] Shuihua Zhao, Tianyu Zhang, Siyuan Ma, and Mingyang Wang. Dandelion optimizer: A nature-inspired metaheuristic algorithm for engineering applications. *Engineering Applications of Artificial Intelligence*, 123:106570, 2023.

[30] Yang Liu, Kun Wang, and Zhu Han. Distributed resource allocation via best response dynamics in mobile edge computing. *IEEE Transactions on Wireless Communications*, 21(4):2369–2384, 2022.

[31] Yiran Chen, Minghui Zhu, and Feng Liu. Fictitious play for delay-sensitive task offloading in mobile edge computing. *IEEE/ACM Transactions on Networking*, 31(2):987–1002, 2023.

[32] Shun Li, Haibo Ge, and Xuelong Zhang. Replicator dynamics for energy-efficient cooperation in iot networks. *IEEE Internet of Things Journal*, 10(5):4321–4335, 2023.